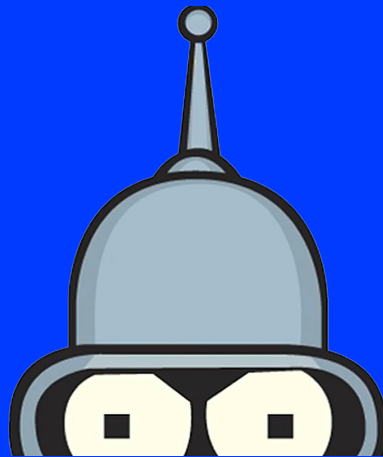


{EPITECH}

ROBOT FACTORY

< ELEMENTARY PROGRAMMING IN C />



ROBOT FACTORY



binary name: robot-factory

language: C

compilation: via Makefile, including re, clean and fclean rules

Authorized functions: (f)open, (f)read, (f)write, (f)close, (l)stat, lseek, fseek, getline, malloc, realloc, free



- ✓ The totality of your source files, except all useless files (binary, temp files, objfiles,...), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.
- ✓ Error messages have to be written on the error output, and the program should then exit with the 84 error code (0 if there is no error).

Lore

You have brilliantly navigated the preparation of your lab, secured your most precious plans, and organized your workspace into a true fortress of efficiency. Now, the time has come to bring your ambitions to life. Before you lies the most exhilarating challenge yet: the Robot Factory.

In this new adventure, you'll step into the shoes of a true robotics engineer, but not just any engineer. Your mission is to transform theoretical plans, written in assembly language, into combat-ready machines. These robots will be your champions in the arena of the grand Core-war tournament.

Assembly language, with its precise instructions and low-level operations, is the ancient tongue of machines. You must master it to code the heart of your robots. But that's only the first step. Every line of code, every instruction, must be translated into bytecode - the universal language of our mechanical fighters. It's in this complex alchemy that the power and soul of your future champion reside.

Imagine your plans as ancient scrolls, holding the secrets of legendary warriors. Your task is to decipher these scrolls and transmute them into living force. This process requires a deep understanding of assembly and a mastery of transcription into bytecode, a skill that separates the true masters of robotics from the novices.

The project

Objectives

The Corewar tournament is a game in which several programs called **Champions will fight to be the last one alive.** Corewar is actually a computer simulation in which processors will share memory to run on.

The project is based on a virtual machine in which the champions will fight for memory by all possible means in order to win, the last champion being able to signal that it is alive wins the game. In other words, the last champion to execute the **live** instruction is declared winner.

The Robot Factory project will give you the ability to transform champion plans, files written in **assembly**, into tournament-ready champions, files written in **bytecode**.



Search “corewars” and “redcode” on the Internet...

The different parts

The project is divided into three separate parts, with only one to develop:

- ✓ Champions (given, you don't *have* to write any)
The champions are files written in an assembly language specific to our virtual machine. The file is made of instructions that the champion must follow, it is its line of conduct. It is this file that specifies when the champion must attack, defend itself or announce that it is still alive. Examples of champions are given in the attachments.
- ✓ The Assembler (Robot Factory)
The purpose of this program, like a compiler, is to transcribe the champions (the .s files) from a human readable language into a language that the Corewar tournament program can understand. It will therefore be necessary to understand the assembly language in order to translate it byte by byte.
- ✓ The Virtual Machine (Corewar, the next project)

The virtual machine comes only after the development of the assembly part because it will ask you to execute the instructions of each of the champions and thus... to understand the machine language that you just translated! The virtual machine is thus a program which will provide a memory zone that the champions can share and fight in. It is important that the machine properly executes each instruction of each champion in order and waiting for the right amount of cycles until the end of the given time, or only one champion remains.

op.c and op.h

To get to the end of the Corewar and for each of you to develop champions with similar instructions and architecture, two files are made available to you: [op.c](#) and [op.h](#).
You will have to integrate them into your rendering directory.



All values written in **UPPERCASE** in this subject are variables obtainable in [op.c](#) or [op.h](#).



The coding style will be checked on all the files that you deliver, including the currently non-compliant [op.c](#) and [op.h](#).
This means you may modify those files according to your needs.

Champions

Here's what a champion (a .s file) might look like:

```
Terminal
$> cat jon.s
.name "Jon Snow"
.comment "Winter is coming"

sti r1, %:crow, %1
crow:live %234
ld %0, r3
zjmp %:crow
```

The header

The header contains the informations that will be used to display champions informations during tournament.

When a champion is compiled into a .cor file, these informations are stored in a binary header. This header starts with a **magic number** (COREWAR_EXEC_MAGIC), which is a fixed value used by the virtual machine to recognize a valid champion file before reading its name, size and comment.

- ✓ A name

```
.name "Jon Snow"
```

- ✓ A description

```
.comment "winter is coming"
```



There can be comments anywhere as long as the comment starts with a '#'.

The body

The body is a succession of lines which are for each of them a different instruction. In the example we have 4 distinct instructions like `sti r1, %:crow, %1`. These instructions are themselves decomposable into 3 sub-parts:

- ✓ There are so-called instruction codes (called opcode), which allow to define which instruction to execute.
 - `sti`, `live`, `ld`, `zjmp`

- ✓ And the parameters that go with them. Each instruction depends on parameters to be used.
 - `r1, %:crow, %1`
 - `%234`
 - `ld %0, r3`
 - `%:crow`
- ✓ But there are also labels. Followed by the **LABEL_CHAR** character (here, `:`). Labels can be any character strings that is composed exclusively of elements from the **LABEL_CHARS** string. It serves as a coordinate point in the instructions of a champion. It can be used, for instance, to jump to that position and execute instructions from there, or to store data relatively to that point in the code.
 - `crow:`

The instructions

Champions have the ability of executing any of 16 instructions (detailed and established in the `op.c` source file provided). You can find details about those in the appendix "Corewar-Instructions" PDF provided alongside this subject.

Don't worry if you don't yet understand everything written there; a lot of this information will actually come in handy during the Corewar project, not necessarily now!



It is not your job to write your own champions! It could be a nice bonus and help you understand their behavior but champions are already provided as attachments to the project.



There are things there you may not understand. The **PC**, **IDX_MOD**, ..., since these are the effects of instructions, they could be interesting elements for Corewar, but maybe not now. Still, you should keep it under your hat in the future, you never know.

The assembler

```
Terminal
$> ./robot-factory -h
USAGE
./robot-factory file_name[.s]
DESCRIPTION
file_name file in assembly language to be converted into file_name.cor, an executable in the Virtual Machine.
```

Now that you know what a champion is made of, you will have to transcribe them into a language that the

virtual machine can understand, i.e. into machine code.
To do this, you will have to translate line per line.

Parameters

As seen before, the structure of an instruction is like so: **<opcode> <parameters>**. It seems that the opcode is understood, it is the instruction to be executed, but what do the parameters that follow mean? An instruction can have from 0 to **MAX_ARGS_NUMBER** parameters, separated by commas. These parameters can be of 3 different types:

- ✓ Register
Each champion will have to **REG_NUMBER** registers in which it can store integers (**r1** to **rREG_NUMBER**). Besides, the first register (**r1**) will contain the identifier of the champion in question.
- ✓ Direct
The **DIRECT_CHAR** character, followed by a value or a label (preceded by **LABEL_CHAR**), which represents the direct value. For instance, **%4** or **:%label**
- ✓ Indirect
A value or a label (preceded by **LABEL_CHAR**), which represents the value that is found at the parameter's address (in relation to **PC**).

Do you now understand `sti r1, %:crow, %1`? The instruction `sti` is executed with the direct value of `:crow` and `1`.



For your interest, a register is a memory location internal to a processor and is the fastest memory in a computer.

Example

Before continuing further, here is what the champions should look like once transcribed.



Using the shell command `hexdump -C <file>` will come to you as frantically as the *revelio* spell in Hogwarts Legacy


```
Terminal
$> ./robot-factory jon.s && hexdump -C jon.cor
00000000 00 ea 83 f3 4a 6f 6e 20 53 6e 6f 77 00 00 00 00 |....Jon Snow....|
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00000080 00 00 00 00 00 00 00 00 00 00 16 57 69 6e 74 |.....Wint|
00000090 65 72 20 69 73 20 63 6f 6d 69 6e 67 00 00 00 00 |er is coming....|
000000a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00000890 0b 68 01 00 07 00 01 01 00 00 00 ea 02 90 00 00 |.h.....|
000008a0 00 00 03 09 ff f4 |.....|
000008a6
```

To read this bytecode correctly, do not assume that every instruction is encoded as [opcode][coding byte][parameter]. The `live`, `zjmp`, `fork` and `lfork` instructions have no coding byte, and indexed direct values are written on `IND_SIZE` bytes.

In this example, the final bytes `09 ff f4` are therefore `zjmp %:crow`: `09` for the opcode, then `ff f4` for its direct indexed parameter.

- ✓ `0b 68 01 00 07 00 01` corresponds to `sti r1, %:crow, %1`
- ✓ `01 00 00 00 ea` corresponds to `live %234`
- ✓ `02 90 00 00 00 00 03` corresponds to `ld %, r3`
- ✓ `09 ff f4` corresponds to `zjmp %:crow`

Transcription

With hexdump, you have the ability to see each of the bytes transcribed by the assembly. For example you can see that the 4th byte `4a` is the hexadecimal value of the character `J`, the same `J` as in `Jon Snow`! This means two things:

- ✓ It is possible to take byte after byte a `.cor` file with its original `.s` file to understand how the transcription is made (to understand the particular cases it is very practical!)
- ✓ Before transcribing the instructions, the header (with name and comment) must be transcribed using the `header_t` structure provided in `op.h`.

Each instruction is coded through 3 elements:

- ✓ **the instruction code** (opcode)
- ✓ **the coding byte**, which is a value to describe the type of the parameters that follow (after all, once in machine code, how do you know what type the parameters are?) The byte coding will not be present for the `live`, `zjmp`, `fork` and `lfork` instructions.
- ✓ **the parameters**

But how to write them?

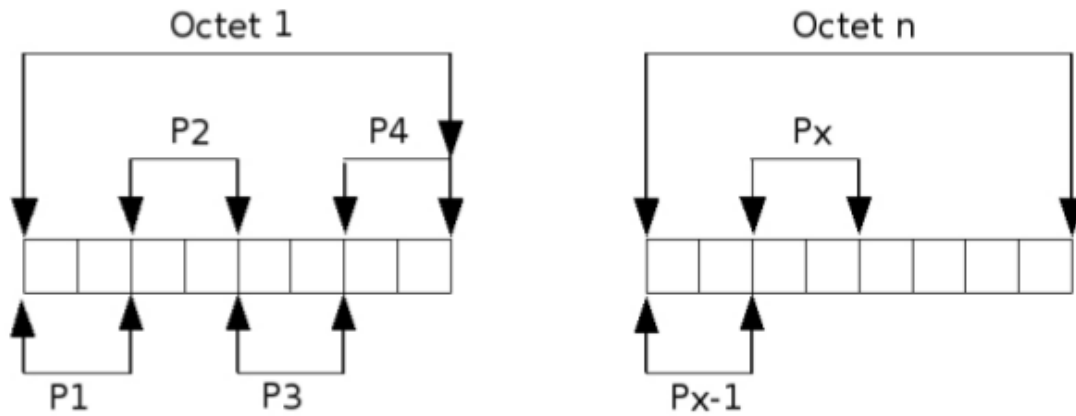
Instruction code

The instruction code can be found in `op_tab` in `op.h`. This instruction code is written on a single byte. This will give `0x0b` for the `sti` instruction in our example.

Coding byte

The byte coding is calculated according to the following parameters. In a byte we will fill each of its 8 bits from the nature of the parameters. Bits 1 and 2 will be the type of parameter 1, bits 3 and 4 the type of parameter 2 etc.

- ✓ If the parameter is a register: 01
- ✓ If the parameter is a direct: 10
- ✓ If the parameter is an indirect: 11
- ✓ Otherwise: 00



The whole thing then forms a single byte ready to be written. This will give for example :

- The parameters `r2, 23, %34` will give: `01 11 10 00` and therefore equal: `0x78`
- The parameters `23, 45, %34` will give: `11 11 10 00` and therefore equal: `0xf8`
- The parameters `r1, r3, 34` will give: `01 01 11 00` and therefore will be equal to: `0x5c`

Parameters

The parameters are transcribed one by one by writing their value directly on a number of bytes equivalent to their type.

- ✓ The value is written (in hexadecimal) on 1 byte for a register
- ✓ The value is written (in hexadecimal) on **DIR_SIZE** bytes for a direct
- ✓ The value is written (in hexadecimal) on **IND_SIZE** bytes for indirect

Using the examples from before:

- The parameters `r2, 23, %34` will give: `0x02, 0x00 0x17` and `0x00 0x00 0x00 0x22`
- The parameters `23, 45, %34` will give: `0x00 0x17, 0x00 0x2d` and `0x00 0x00 0x00 0x22`
- The parameters `r1, r3, 34` will give: `0x01, 0x03` and `0x00 0x22`

What happened when the parameter is a label like `:%crow`? You should let's compare `.s` and `.cor` file to see the result!



Those examples are only valid when none of the parameters are indexes

Did you say indexes ?

Instructions will sometime describe the parameters they take as [indexes](#) . This does not change the rules for the description of the parameters type but the size of the parameter itself might change.

Indexes size will always be **IND_SIZE** bytes, even when the parameter is a direct.
Read the Instructions section carefully to identify the function that have indexes as parameters.



The virtual machine for the tournament is BIG ENDIAN (like the Sun and unlike the i386). This changes the order of storage, and thus perhaps the way you should transcribe your information

Conclusion

For the rest, think about it and read [op.h](#) and [op.c](#).

Your teaching assistants have also been through this, so don't be surprised if one of them looks at you wide-eyed when you ask them for help on certain bytes (spoiler: they'll know how to answer you, but might like to let you do a bit of thinking - that's the point of the project!).

Please verify that your programs are completely up to the coding style.



If you get this far, go back to page 2.



If you've arrived here again, go back to page 2 - it never hurts to reread the subject!



Here again? No but really, I assure you that rereading the subject one more time won't do you any harm.

v1

{EPITECH}